Claude Code in Action

Writing better tickets, code & tests — live

Marnus Coetzee | WBC

What We'll Cover

1. **Writing Insightful Tickets** — ~10 min
   From vague requests to precise, testable work items

2. **Writing & Debugging Code** — ~10 min
   Real-world coding with Claude as a thinking partner

3. **Skills & Commands** — ~10 min
   Extending Claude Code into your daily workflow

01

Writing Insightful Tickets

Turning vague requests into precise, testable, developer-ready work items

The Ticket Everyone Dreads

Before

Title: Fix the report

Description:
The report is broken. It doesn't show the right numbers. Please fix.

Acceptance Criteria:
Report works correctly.

After (with Claude Code)

Title: Report shows stale data after manual refresh

Description:
Cache invalidation fails when...

Acceptance Criteria:
- Report refreshes within 5s
- Edge case: concurrent users
- Regression test added

Claude Code reads your codebase, understands context, and writes tickets that devs can actually action.

How Claude Code Writes Better Tickets

Step 1

Reads Your Codebase

Understands the existing architecture, models, and data flow

Step 2

Identifies Root Cause

Pinpoints the actual problem, not just the symptom

Step 3

Writes Structured Tickets

Clear title, context, acceptance criteria, edge cases

Step 4

Adds Testability

Includes how to verify the fix works — QA-ready from day one

Live Demo

Taking a vague bug report and turning it into a fully structured ticket with acceptance criteria, edge cases, and test scenarios

02

Writing & Debugging Code

Claude Code as a thinking partner, not just an autocomplete

Why This Isn't Just Another Copilot

Traditional AI Assistants	Claude Code
✗ Autocomplete one line at a time	✓ Reads your entire codebase
✗ No awareness of your project	✓ Understands architecture & patterns
✗ Can't read existing code	✓ Writes, edits, and runs code directly
✗ No debugging capability	✓ Debugs by reading errors + context
✗ Copy-paste from chat windows	✓ Iterates until the solution works

The Development Workflow

Describe the task → Claude reads your code → Writes a solution → Tests & debugs → Working code

Example prompt:

"The monthly report shows stale data after refresh. Find the caching bug, fix it, and write a regression test."

Live Demo

Finding a bug, understanding the code, writing a fix,
and generating tests — all in one conversation

03

Skills & Commands

Making Claude Code work the way your team works

Skills = Reusable Expert Knowledge

Skills are instruction files that teach Claude Code how to do specific tasks consistently, following your team's standards every time.

Ticket Writing Skill

Enforces your team's ticket template, adds acceptance criteria, references related code

Code Review Skill

Checks for your coding standards, security patterns, and naming conventions

Test Generation Skill

Creates tests matching your framework, coverage requirements, and edge case patterns

Documentation Skill

Generates docs in your format — API specs, architecture decisions, runbooks

Slash Commands = One-Click Workflows

Type a slash command and Claude Code runs a predefined workflow instantly.

`/review-pr`	Reviews a pull request against your team's standards
`/write-spec`	Generates a full PRD from a feature idea
`/debug`	Investigates an error with full codebase context
`/test`	Generates comprehensive tests for a module
`/document`	Creates documentation in your team's format

You can create custom commands tailored to your team's specific workflows

Live Demo

Creating a custom skill for your team, running a slash command, and showing how it all connects to your daily workflow

Everyone Benefits — Not Just Developers

Developers

Write code faster, debug smarter, generate tests, automate repetitive tasks

Business Analysts

Write precise tickets, generate specs, create documentation, validate requirements against code

QA Engineers

Generate test plans, write automated tests, create edge case scenarios, validate acceptance criteria

Skill vs Command — The Difference

Skill = The Knowledge

A markdown instruction file that teaches Claude Code how to do something — persistent expert knowledge that shapes its behaviour every time that topic comes up.

Think of it as:
A playbook or standard operating procedure that Claude follows automatically.

Command = The Trigger

A shortcut you type (like /review-pr) that triggers a specific workflow instantly — a button that runs a predefined sequence of steps.

Think of it as:
A shortcut key or macro that kicks off a workflow with one action.

A skill is the knowledge. A command is the trigger. Commands often use skills under the hood.

Questions Developers Will Ask

Does it send my code to the cloud?

Yes, code context is sent to Anthropic's API for processing, but Anthropic does not train on your data. Enterprise plans offer additional data governance.

Can it work with our specific tech stack?

Claude Code is language and framework agnostic — it reads your actual files, so it adapts to whatever you're using.

Will it just generate buggy code I have to fix?

It iterates. You can push back, ask it to debug its own output, and it will run tests to verify before finishing.

How is this different from GitHub Copilot?

Copilot autocompletes lines. Claude Code reads your entire codebase, understands architecture, writes across files, and executes commands.

Questions BAs Will Ask

I'm not technical — can I still use this?

Absolutely. You describe what you need in plain English. Claude Code handles the technical translation. It's like having a developer sitting next to you.

Can it read our existing documentation?

Yes. Point it at your codebase, wiki exports, or spec files and it will use that context to write better tickets and requirements.

How do I know the output is accurate?

Claude Code references actual code and data models. It shows its reasoning, so you can verify. Always review — it's a partner, not a replacement.

Will this replace BAs?

No. It amplifies your expertise. You still make the decisions — Claude Code just eliminates the tedious formatting, research, and translation work.

Questions QA Will Ask

Can it write automated tests, not just manual ones?

Yes. It writes unit tests, integration tests, and E2E tests in your framework — NUnit, xUnit, Playwright, Selenium, whatever you use.

How does it know what edge cases to test?

It reads the actual code paths, identifies boundary conditions, null checks, race conditions, and error handling — things humans often miss on first pass.

Can it help with regression testing?

Absolutely. Describe what changed, and it generates targeted regression tests that cover the affected code paths specifically.

Does it understand our test data requirements?

You can teach it via skills. Create a skill with your test data patterns, naming conventions, and environment setup — it follows them every time.

The Tough Questions (Be Ready)

What about hallucinations / wrong answers?

It happens. That's why you verify. But Claude Code can run its own code and check its work — it catches most errors through iteration. Treat it like a junior dev: review everything.

What's the cost?

Anthropic offers team and enterprise plans. The ROI is measured in hours saved per developer per week — typically 5-10 hours reclaimed.

Is our code safe / what about IP?

Anthropic doesn't train on your data. Enterprise plans include SOC 2 compliance and data retention controls. Check your organisation's AI policy.

Why not just use ChatGPT / free tools?

Claude Code runs in your terminal, reads files, and iterates autonomously. ChatGPT is a chat window you copy-paste into. Different paradigm.

The SDLC: 1 B.C. vs 1 A.C.

(Before Claude vs After Claude)

1 B.C. (Before Claude)		1 A.C. (After Claude)
2-3 sprint meetings Requirements & Refinement — ~2 weeks	1	Claude + BA in one session Requirements & Refinement — ~2 hours
BA writes, dev rewrites, QA adds criteria Ticket Writing — ~3 days	2	Claude drafts complete tickets from epic Ticket Writing — ~30 min
One dev, one feature, sequential Development — ~1 week	3	3 worktree agents in parallel Development — ~1 day
Wait for reviewer, back-and-forth Code Review — ~2 days	4	Code Review plugin + human approval Code Review — ~2 hours
Manual test plan, write tests, run them Testing — ~3 days	5	Claude generates tests from AC Testing — ~3 hours
"We'll do it later" (never) — ~0 days	6	Auto-generated with the code — ~0 effort
Total: ~4 weeks		**Total: ~3 days**

04

The Wow Factor

Plugins, Code Review, and Parallel Worktrees — where it gets serious

Plugins — Install Superpowers in Seconds

Plugins are installable bundles of skills, commands, and tool integrations.
One click to install, instantly available across your team.

Skill Creator

Build custom skills by describing what you want. Claude writes the SKILL.md, tests it, and optimises it — you don't write a single line of config.

Code Review

Dispatches parallel agents to review PRs, find bugs, filter false positives, and post ranked in-line comments directly on GitHub.

Feature Dev

7-phase workflow: requirements, codebase exploration, architecture, implementation, testing, review, and documentation — all automated.

340+ plugins and 1,300+ skills available — or build your own in minutes.

Code Review — AI That Actually Finds Bugs

1

PR Opens

Automatically triggered or via @claude review comment

2

Parallel Agents

Multiple agents scan for different bug categories simultaneously

3

Verify & Filter

False positives are eliminated — only real issues remain

4

Ranked Report

Bugs posted as in-line comments, ranked by severity

What makes this different from other review tools:

- It reads the full codebase context, not just the diff
- It understands intent — catches logical bugs, not just style issues
- Posts directly to GitHub as actionable in-line comments

Worktrees — Parallel Agents, Zero Conflicts

Each agent gets its own isolated copy of the repository.
They work in parallel without stepping on each other's changes.

Your Main Repo

Agent 1: Fix auth bug

Agent 2: Add feature

Agent 3: Write tests

No context bleed between agents

Auto-cleanup when done

Start with --worktree or -w flag

The Full Picture — A Day with Claude Code

09:00	Write tickets from vague requirements	BA
09:30	Spin up 3 worktree agents for parallel dev	Dev
10:00	PR auto-reviewed by Code Review plugin	Auto
10:15	Generate test plans from acceptance criteria	QA
10:30	Custom skill writes regression tests	QA
11:00	Feature shipped with docs, tests, and review	Team

From ticket to shipped feature in a single morning — across all three roles.

Start Using Claude Code Today

Don't just code faster — think better, document smarter, and ship with confidence.

10x
Faster ticket writing

5x
Faster debugging

∞
Custom skills possible

Questions?

Marnus Coetzee

MarnusC@webuycars.co.za